

PRACTICAL WEEK-1

Task1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user by accepting a Linux command as input, creating a child process using the fork() system call, executing the command in the child process using an appropriate exec() system call, allowing the parent process to wait for the child using wait(), and displaying the Process ID (PID) of both the parent and child processes.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char command[100];
    char *args[20];
    char *token;
    int i = 0;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline
    command[strcspn(command, "\n")] = '\0';

    // Split command into arguments
    token = strtok(command, " ");

    while (token != NULL && i < 19) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command...\n");

        execvp(args[0], args);
    }
}
```

```
execvp(args[0], args);

perror("exec failed");
exit(1);
} else {
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);
    printf("Parent waiting for child...\n");

    wait(NULL);

    printf("Child process completed.\n");
    printf("Parent process completed.\n");
}

return 0;
}
```

Output:

```
anuan@Anushka:~$ nano command_execution.c
anuan@Anushka:~$ gcc command_execution.c -o command_execution
anuan@Anushka:~$ ./command_execution
Enter a Linux command: ls

Parent Process:
Parent PID : 810
Child PID : 811

Child Process:
Child PID : 811
Parent PID : 810
File1.txt          fork_exec.c        process.c.save.1
File2.txt          fork_exec.c.save   process_fork_states
OSSP               fork_exec.r         process_fork_states.c
Practical          hardware_os         process_states
Skill              hardware_os.c       process_states.c
client             hello               producer_consumer
client.c           hello.c             producer_consumer.c
command_execution  history.c           product.c.save
command_execution.c history_input.c     program.c
command_executor  lex_parser          ps.c
command_executor.c lex_parser.c        ps.c
constant          main_loop           pt.c
constant.c        main_loop.c         pmd
counter           memory              sample.txt
counter_mutex     memory.c            server
counter_mutex.c   memory.c.save       server.c
cow               parser.c            server_fifo
cow.c             parser_lexer.c       session12_task1
deadlock.c.save   parser_lexer.c.save session12_task1.c
deadlock.c         pc.c.save           signals
destination.txt   pd.c                signals.c
double_quotes     pl.c.save           single_quotes
double_quotes.c   pipe_exec           single_quotes.c
double_quotes.c.save pipe_exec.c         source.txt
file_copy         pipeline            state_monitor.c
file_copy.c       pipeline.c          task2
file_copy.c.save  pl.c               task2.c.save
filecopy          pl.c.save           wait_waitpid
filecopy2         process            wait_waitpid.c
fork_exec         process.c           waitpid_demo
                  process.c           waitpid_demo.c

Parent: Child process has completed.
Child exit status: 0
anuan@Anushka:~$
```

Observation: I learned how the Linux operating system creates a child process using fork() and executes a Linux command using the execvp() system call. I also understood how the parent process waits for the child process using wait() and how PID and PPID help identify the relationship between parent and child processes.

Task2: Using Linux terminal commands such as uname, lscpu, lsblk, ps, and top, investigate the relationship between hardware resources and operating system services. Prepare a

report explaining how the operating system abstracts and manages CPU, memory, storage, and I/O devices.

Output:

Using command `uname -a & lscpu`

```
anuan@Anushka:~$ uname -a
Linux Anushka 6.18.31.2-microsoft-standard-WSL2 41 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
anuan@Anushka:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                16
On-line CPU(s) list:   0-15
Vendor ID:              GenuineIntel
Model name:             13th Gen Intel(R) Core(TM) i7-13620H
CPU family:            6
Model:                 186
Thread(s) per core:    2
Core(s) per socket:    8
Socket(s):             1
Stepping:               3
BogoMIPS:              5836.00
Flags:                  fpu vme de pae tsc mtr pae mce cmx apic xapic xapic
                        rcp mce cdev pat pse36 clflush mmx fxsr sse
                        sse2 ss ht syscall nx pdpe1gb rdtscp lm constant
                        tsc rep_good nopl stompology tsc_reliable n
                        ontop_tsc cpuid tsc_lunom_freq pni pclmuldq
                        vme dca3 fma cldx pcid sha3_1 sha3_2 sha3c w
                        oves popcnt tsc_deadline_timer sse4_1 sse4_2
                        16c rdrand hypervisor lahf_lm smm 3dnowprefetch
                        h sand ibrs ibpb stibp ibrs_enhanced tpr_shadow
                        w ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx
                        2 smep bmi2 erms drpcid rdtscp mba rtmcp clfu
                        shopt clwb sha_ni xsaveopt xsavec sgatb1 xsave
                        avx_vnni vmx umip waitpkg gfni vaes vpclmul
                        lqd rdtcd mwaitx mwaitx64 fcrmsd clear err
                        ialize ibt flush_l1d arch_capabilities

Virtualization features:
  Virtualization:       VT-x
  Hypervisor vendor:    Microsoft
  Virtualization type:  Full
Caches (sum of all):
  L1d:                  384 KiB (8 instances)
  L1i:                  256 KiB (8 instances)
  L2:                   30 MiB (8 instances)
  L3:                   24 MiB (1 instance)
NUMA:
  NUMA node(s):         1
  NUMA node0 CPU(s):   0-15
Vulnerabilities:
  Gather data sampling: Not affected
  Disruptive:           Not affected
  Indirect target selection: Not affected
  Itlb multihit:        Not affected
  L1tf:                 Not affected
  Mds:                  Not affected
  Meltdown:             Not affected
  Mmio stale data:      Not affected
  Old microcode:        Not affected
  Reg file data sampling: Mitigation: Clear Register File
  Retbleed:             Mitigation: Enhanced IBRS
  Spec rstack overflow: Not affected
  Spec store bypass:    Mitigation: Speculative Store Bypass disabled
                        via prctl
  Spectre v1:           Mitigation: usercopy/swapgs barriers and __user
                        pointer sanitization
  Spectre v2:           Mitigation: Enhanced / Automatic IBRS; IBPB co
                        nditional; PRRSDB=IBRS SW sequence; BHI BHI_DT
                        2.5
  Srbds:               Not affected
  Taa:                  Not affected
  Taa async abort:     Not affected
  Vmcs:                 Not affected
```

Using command `lsblk&ps`

```
anuan@Anushka:~$ lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.4M  1 disk
sdc   8:32   0      2G  0 disk [SWAP]
sdd   8:48   0      1T  0 disk /mnt/wslg/distro
/

anuan@Anushka:~$ ps
  PID TTY          TIME CMD
  752 pts/0        00:00:00 bash
  864 pts/0        00:00:00 ps
anuan@Anushka:~$
```

Use command `top`:

```

aman@minisha:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 24184 15364 ?        Ss   04:43   0:00 /sbin/init
root         2  0.0  0.0   3188   2208 hvcd0 S+   04:43   0:00 /init
root         8  0.0  0.0   3188   2044 hvcd0 S+   04:43   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd
19 --log
root        48  0.0  0.2  58428 16728 ?        Ss   04:43   0:00 /usr/lib/systemd/systemd-journald
systemd+   88  0.0  0.1  22940 14608 ?        Ss   04:43   0:00 /usr/lib/systemd/systemd-resolved
root        87  0.0  0.1  35420 12416 ?        Ss   04:43   0:00 /usr/lib/systemd/systemd-udevd
root       165  0.0  0.0   2888   1940 ?        Ss   04:43   0:00 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
root       167  0.0  0.0   4072   3176 ?        Ss   04:43   0:00 /usr/sbin/cron -f -p
message+   170  0.0  0.0   8792   5396 ?        Ss   04:43   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile
--systemd
root       178  0.0  0.3 44892 29868 ?        Ss   04:43   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-ti
iggers
root       182  0.0  0.1  18432   9444 ?        Ss   04:43   0:00 /usr/lib/systemd/systemd-logind
syslog     211  0.0  0.0  226948   5212 ?        Ssl  04:43   0:00 /usr/sbin/rsyslogd -n -iNONE
root       214  0.0  0.0   5892   2740 ttyL    S+   04:43   0:00 /usr/sbin/agetty --noclear --issue-file=/etc/issue:/
etc/issue
root       238  0.0  0.0  28420   6404 ?        S    04:43   0:00 /usr/sbin/chronyd -n -F 1 -x
chrony     247  0.0  0.4 123456 22532 ?        Ssl  04:43   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upg
rade-shut
root       253  0.0  0.0  12892   2340 ?        S    04:43   0:00 /usr/sbin/chronyd -n -F 1 -x
chrony
root       337  0.0  0.0   8640   5596 ?        Ss   04:43   0:00 login -- aman
aman       383  0.0  0.1  22940 13608 ?        Ss   04:43   0:00 /usr/lib/systemd/system --user
aman       385  0.0  0.0  22912   4076 ?        S    04:43   0:00 (sd-pam)
aman       413  0.0  0.0   6136   5088 pts/1  Ss+  04:43   0:00 -bash
root       744  0.0  0.0   3196   1124 ?        Ss   04:54   0:00 /init
root       746  0.0  0.0   3196   1260 ?        S    04:54   0:00 /init
aman       752  0.0  0.0   6268   5608 pts/0  Ss   04:54   0:00 -bash
aman       876  0.0  0.0   7168  4364 pts/0  R+   04:59   0:00 ps aux
aman@minisha:~$

```

Observation: I learned how to use Linux commands to examine system hardware and running processes. I understood how the operating system manages CPU, storage, memory, and processes and provides these resources to applications through system services